Report for the Course on
Cyber-Physical Systems and IoT Security

Reference paper: *"Attack Taxonomies for the Modbus Protocols"*
Authors: Peter Huitsing, Rodrigo Chandia, Mauricio Papa, Sujeet Shenoi — Critical Infrastructure Protection II, Springer, 2008

Student: Pivato Riccardo 2159405, Thomas Rigoni 2158524

1. Objectives

This project implements a Modbus TCP testbed using open-source tools and demonstrates representative attacks from the taxonomy presented in Table 4 of the reference paper. The paper classifies Modbus attacks across four threat categories - Interception, Interruption, Modification, and Fabrication - and four asset classes - Master, Field Device, Network Path, and Message - yielding 28 distinct attack types and 113 total instances.

Our objectives are threefold. First, to build a functional Modbus TCP simulation consisting of a field device (slave), a SCADA master, and an attacker node, all running as isolated Docker containers on a private network. Second, to implement and execute two attacks per threat category - seven attacks in total - covering the full breadth of the taxonomy. Third, to critically evaluate each attack's effectiveness, analyse the root causes that make Modbus TCP vulnerable, and discuss realistic countermeasures.

2. Background

2.1 Modbus TCP Protocol

Modbus is a serial communication protocol developed by Modicon in 1979 and later extended to TCP/IP as Modbus TCP. It is the de-facto standard for industrial field-bus communication and remains in widespread use in power grids, water treatment plants, oil pipelines, and manufacturing facilities.

A Modbus TCP frame consists of a 6-byte MBAP header followed by a PDU. The MBAP header carries a Transaction ID (2 bytes), Protocol Identifier fixed at 0x0000 (2 bytes), Length (2 bytes), and Unit Identifier (1 byte). The PDU begins with a Function Code (FC) that specifies the requested operation: FC03 Read Holding Registers, FC06 Write Single Register, FC16 Write Multiple Registers, among others. Holding registers are 16-bit unsigned integers accessible at addresses 0–65535.

Critically, Modbus TCP has no session establishment, no authentication handshake, and no message integrity check. Any host that can open a TCP connection to port 502 of a slave can read or write any register without presenting credentials.

2.2 Attack Taxonomy (Table 4)

Huitsing et al. (2008) systematically map Modbus attacks to the classic CIA threat model. Table 4 organises 28 attack types (113 instances) across four categories:

Interception - confidentiality violation; the attacker observes traffic or queries the slave without authorisation.

Interruption - availability violation; the attacker prevents the master from receiving valid responses.

Modification - integrity violation; the attacker alters messages in transit, so the master receives falsified data.

Fabrication - authenticity violation; the attacker injects or replays messages, impersonating a legitimate participant.

Each category targets one or more of four asset classes: Master, Field Device, Network Path, and Message. Fabrication has the largest surface - B2 alone has 6 instances and B14 has 15 - because an unauthenticated protocol allows full impersonation of both master and slave. The reference codes follow the paper's convention: 'B' attacks exploit Modbus application-layer properties; 'T' attacks exploit the underlying TCP transport layer.

3. Testbed Setup

3.1 Architecture

The testbed consists of three Docker containers connected on a private bridge network (subnet 172.20.0.0/24). The entire environment is started with a single docker compose up --build command, making the testbed fully reproducible on any machine with Docker Desktop installed.

```
modbus-net  (172.20.0.0/24)
┌──────────────────────────────────────────────────────────────────────┐
│  master (172.20.0.3)  ──Modbus TCP/502──▶  slave (172.20.0.2)        │
│  attacker (172.20.0.4)  Python · pyModbus 3.7.0 · Scapy 2.6.1        │
└──────────────────────────────────────────────────────────────────────┘
```

The complete source code, Dockerfiles, attack scripts, and captured pcap files are publicly available at https://github.com/riccardopivato/modbus-tcp-attacks to allow full reproduction of the experiments described in this report.

3.2 Field Device Simulator (Slave)

The slave is implemented with pyModbus 3.7.0's StartAsyncTcpServer and exposes five holding registers modelling a representative industrial sensor node:

```
HR[0]  Temperature      (×0.1 °C)     initial: 235   →   23.5 °C
HR[1]  Pressure         (×0.1 kPa)    initial: 856   →   85.6 kPa
HR[2]  Flow rate        (×0.1 L/min)  initial: 1020  →  102.0 L/min
HR[3]  Valve status     (0=closed, 1=open)    initial: 0
HR[4]  Supply voltage   (×0.1 V)      initial: 480   →   48.0 V
```

3.3 SCADA Master Simulator

The master is a polling client built with pyModbus's AsyncModbusTcpClient. It issues a FC03 Read Holding Registers request (address 0, count 5) every 2 seconds and prints the decoded sensor values. On connection loss it automatically retries with a 3-second back-off, simulating a real SCADA polling loop.

```
C:\Users\Utente\Desktop\CPS and IOT security\2_project>docker logs --tail 8 modbus-master
[MASTER] Temp (x0.1°C)=235 | Pressure (x0.1 kPa)=856 | Flow (x0.1 L/min)=1020 | Valve (0=closed)=0 | Voltage (x0.1 V)=480
[MASTER] Temp (x0.1°C)=235 | Pressure (x0.1 kPa)=856 | Flow (x0.1 L/min)=1020 | Valve (0=closed)=0 | Voltage (x0.1 V)=480
[MASTER] Temp (x0.1°C)=235 | Pressure (x0.1 kPa)=856 | Flow (x0.1 L/min)=1020 | Valve (0=closed)=0 | Voltage (x0.1 V)=480
[MASTER] Temp (x0.1°C)=235 | Pressure (x0.1 kPa)=856 | Flow (x0.1 L/min)=1020 | Valve (0=closed)=0 | Voltage (x0.1 V)=480
[MASTER] Temp (x0.1°C)=235 | Pressure (x0.1 kPa)=856 | Flow (x0.1 L/min)=1020 | Valve (0=closed)=0 | Voltage (x0.1 V)=480
[MASTER] Temp (x0.1°C)=235 | Pressure (x0.1 kPa)=856 | Flow (x0.1 L/min)=1020 | Valve (0=closed)=0 | Voltage (x0.1 V)=480
[MASTER] Temp (x0.1°C)=235 | Pressure (x0.1 kPa)=856 | Flow (x0.1 L/min)=1020 | Valve (0=closed)=0 | Voltage (x0.1 V)=480
[MASTER] Temp (x0.1°C)=235 | Pressure (x0.1 kPa)=856 | Flow (x0.1 L/min)=1020 | Valve (0=closed)=0 | Voltage (x0.1 V)=480
```

Figure 1 - Normal master polling output showing all five sensor registers.

3.4 Attacker Node

The attacker container has Linux capabilities NET_ADMIN and NET_RAW, which allow Scapy to send crafted raw IP/TCP frames. It also has tcpdump for in-container packet capture. All attack scripts are self-contained Python programs executed via docker exec.

Note on Docker bridge networking: Docker's bridge driver implements a virtual switch. Consequently, the attacker container only receives frames addressed to its own MAC, preventing passive sniffing of master-slave traffic. Attacks that required observing master-slave traffic were adapted accordingly, as described in each section.

4. Interception Attacks

Interception attacks violate confidentiality. The attacker extracts information from the network or from individual devices without authorisation and without necessarily sending malicious commands.

4.1 B9 - Passive Reconnaissance

Description. B9 exploits the fact that Modbus TCP transmits all data in cleartext with no session authentication. An attacker with access to the communication link can observe all register addresses, function codes, and data values simply by reading TCP payloads.

Implementation. The script establishes a TCP connection to 172.20.0.2:502, issues 5 consecutive FC03 Read Holding Registers requests, and prints the raw MBAP bytes alongside the decoded sensor values - identical information to what a wire-tap or Wireshark capture would reveal to a passive observer on the link.

```
Amministratore: Prompt dei comandi
C:\Users\Utente\Desktop\CPS and IOT security\2_project>docker exec modbus-attacker python /app/interception_passive.py
[B9] Passive Reconnaissance — connecting to slave (NO authentication required)
[B9] Target: 172.20.0.2:502  unit_id=1

[B9] Cycle 1  —  FC03 Read Holding Registers
   REQ  → 172.20.0.2:502   unit=1  start_addr=0  count=5
   REQ  bytes: 00 01 00 00 00 06 01 03 00 00 00 05
   RSP  ← 172.20.0.2:502   unit=1  register_values=[235, 856, 1020, 0, 480]
   RSP  bytes: 00 01 00 00 00 0D 01 03 0A 00 EB 03 58 03 FC 00 00 01 E0
   Decoded sensor state:
     HR[0]  Temperature  (x0.1 °C) = 235
     HR[1]  Pressure     (x0.1 kPa) = 856
     HR[2]  Flow rate    (x0.1 L/min) = 1020
     HR[3]  Valve status (0=closed, 1=open) = 0
     HR[4]  Supply voltage (x0.1 V) = 480

[B9] Cycle 2  —  FC03 Read Holding Registers
   REQ  → 172.20.0.2:502   unit=1  start_addr=0  count=5
   REQ  bytes: 00 02 00 00 00 06 01 03 00 00 00 05
   RSP  ← 172.20.0.2:502   unit=1  register_values=[235, 856, 1020, 0, 480]
   RSP  bytes: 00 02 00 00 00 0D 01 03 0A 00 EB 03 58 03 FC 00 00 01 E0
   Decoded sensor state:
     HR[0]  Temperature  (x0.1 °C) = 235
     HR[1]  Pressure     (x0.1 kPa) = 856
     HR[2]  Flow rate    (x0.1 L/min) = 1020
     HR[3]  Valve status (0=closed, 1=open) = 0
     HR[4]  Supply voltage (x0.1 V) = 480

[B9] Cycle 3  —  FC03 Read Holding Registers
   REQ  → 172.20.0.2:502   unit=1  start_addr=0  count=5
   REQ  bytes: 00 03 00 00 00 06 01 03 00 00 00 05
   RSP  ← 172.20.0.2:502   unit=1  register_values=[235, 856, 1020, 0, 480]
   RSP  bytes: 00 03 00 00 00 0D 01 03 0A 00 EB 03 58 03 FC 00 00 01 E0
   Decoded sensor state:
     HR[0]  Temperature  (x0.1 °C) = 235
     HR[1]  Pressure     (x0.1 kPa) = 856
     HR[2]  Flow rate    (x0.1 L/min) = 1020
     HR[3]  Valve status (0=closed, 1=open) = 0
     HR[4]  Supply voltage (x0.1 V) = 480

[B9] Cycle 4  —  FC03 Read Holding Registers
   REQ  → 172.20.0.2:502   unit=1  start_addr=0  count=5
   REQ  bytes: 00 04 00 00 00 06 01 03 00 00 00 05
   RSP  ← 172.20.0.2:502   unit=1  register_values=[235, 856, 1020, 0, 480]
   RSP  bytes: 00 04 00 00 00 0D 01 03 0A 00 EB 03 58 03 FC 00 00 01 E0
   Decoded sensor state:
     HR[0]  Temperature  (x0.1 °C) = 235
     HR[1]  Pressure     (x0.1 kPa) = 856
     HR[2]  Flow rate    (x0.1 L/min) = 1020
     HR[3]  Valve status (0=closed, 1=open) = 0
     HR[4]  Supply voltage (x0.1 V) = 480

[B9] Cycle 5  —  FC03 Read Holding Registers
   REQ  → 172.20.0.2:502   unit=1  start_addr=0  count=5
   REQ  bytes: 00 05 00 00 00 06 01 03 00 00 00 05
   RSP  ← 172.20.0.2:502   unit=1  register_values=[235, 856, 1020, 0, 480]
   RSP  bytes: 00 05 00 00 00 0D 01 03 0A 00 EB 03 58 03 FC 00 00 01 E0
   Decoded sensor state:
     HR[0]  Temperature  (x0.1 °C) = 235
     HR[1]  Pressure     (x0.1 kPa) = 856
     HR[2]  Flow rate    (x0.1 L/min) = 1020
     HR[3]  Valve status (0=closed, 1=open) = 0
     HR[4]  Supply voltage (x0.1 V) = 480

[B9] Reconnaissance complete — attacker has full process visibility.
[B9] All data transmitted in cleartext; no credentials required.
```

Figure 2 - B9 output: raw MBAP bytes and decoded sensor values for cycles 1–3.

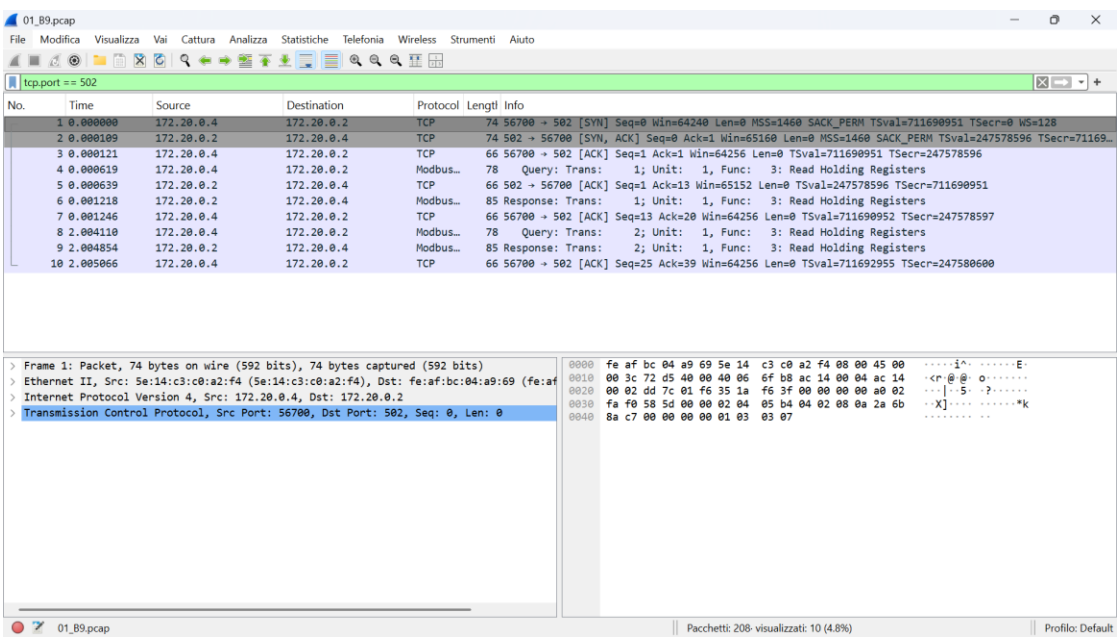

Figure 3 - Wireshark capture of B9 traffic: FC03 queries and responses in cleartext.

Analysis. B9 requires only network reachability to port 502 - no exploit, no vulnerability, no credentials. An attacker who has reached the OT network can fully map the process state in seconds. This information directly enables all subsequent attack categories: knowing the register layout, polling rate, and normal value ranges is sufficient to craft convincing falsified payloads.

4.2 B6 - Modbus Network Scanning

Description. B6 extends passive reconnaissance to active device discovery. The attacker probes all valid Modbus unit IDs (1–247) to enumerate which field devices are present and to read their register maps.

Implementation. The script iterates unit IDs 1-10, issues a FC03 Read Holding Registers request (address 0, count 10) for each, and reports responding devices with their full register dump and labelled sensor values.

```
Amministratore: Prompt dei comandi
C:\Users\Utente\Desktop\CPS and IOT security\2_project>docker exec modbus-attacker python /app/interception_scan.py
[B6] Modbus Network Scan → 172.20.0.2:502
[B6] Probing unit IDs 1-10, reading HR[0-9]

[B6] Unit   1 FOUND — raw HR values: [235, 856, 1020, 0, 480, 0, 0, 0, 0, 0]
        Temperature (x0.1 °C) = 235
        Pressure (x0.1 kPa) = 856
        Flow (x0.1 L/min) = 1020
        Valve status = 0
        Supply voltage (x0.1 V) = 480

[B6] Unit   2 FOUND — raw HR values: [235, 856, 1020, 0, 480, 0, 0, 0, 0, 0]
        Temperature (x0.1 °C) = 235
        Pressure (x0.1 kPa) = 856
        Flow (x0.1 L/min) = 1020
        Valve status = 0
        Supply voltage (x0.1 V) = 480

[B6] Unit   3 FOUND — raw HR values: [235, 856, 1020, 0, 480, 0, 0, 0, 0, 0]
        Temperature (x0.1 °C) = 235
        Pressure (x0.1 kPa) = 856
        Flow (x0.1 L/min) = 1020
        Valve status = 0
        Supply voltage (x0.1 V) = 480

[B6] Unit   4 FOUND — raw HR values: [235, 856, 1020, 0, 480, 0, 0, 0, 0, 0]
        Temperature (x0.1 °C) = 235
        Pressure (x0.1 kPa) = 856
        Flow (x0.1 L/min) = 1020
        Valve status = 0
        Supply voltage (x0.1 V) = 480

[B6] Unit   5 FOUND — raw HR values: [235, 856, 1020, 0, 480, 0, 0, 0, 0, 0]
        Temperature (x0.1 °C) = 235
        Pressure (x0.1 kPa) = 856
        Flow (x0.1 L/min) = 1020
        Valve status = 0
        Supply voltage (x0.1 V) = 480

[B6] Unit   6 FOUND — raw HR values: [235, 856, 1020, 0, 480, 0, 0, 0, 0, 0]
        Temperature (x0.1 °C) = 235
        Pressure (x0.1 kPa) = 856
        Flow (x0.1 L/min) = 1020
        Valve status = 0
        Supply voltage (x0.1 V) = 480

[B6] Unit   7 FOUND — raw HR values: [235, 856, 1020, 0, 480, 0, 0, 0, 0, 0]
        Temperature (x0.1 °C) = 235
        Pressure (x0.1 kPa) = 856
        Flow (x0.1 L/min) = 1020
        Valve status = 0
        Supply voltage (x0.1 V) = 480

[B6] Unit   8 FOUND — raw HR values: [235, 856, 1020, 0, 480, 0, 0, 0, 0, 0]
        Temperature (x0.1 °C) = 235
        Pressure (x0.1 kPa) = 856
        Flow (x0.1 L/min) = 1020
        Valve status = 0
        Supply voltage (x0.1 V) = 480

[B6] Unit   9 FOUND — raw HR values: [235, 856, 1020, 0, 480, 0, 0, 0, 0, 0]
        Temperature (x0.1 °C) = 235
        Pressure (x0.1 kPa) = 856
        Flow (x0.1 L/min) = 1020
        Valve status = 0
        Supply voltage (x0.1 V) = 480

[B6] Unit  10 FOUND — raw HR values: [235, 856, 1020, 0, 480, 0, 0, 0, 0, 0]
        Temperature (x0.1 °C) = 235
        Pressure (x0.1 kPa) = 856
        Flow (x0.1 L/min) = 1020
        Valve status = 0
        Supply voltage (x0.1 V) = 480

============================================================
[B6] Scan complete — 10 unit(s) found: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[B6] Attacker now has a full register map of the ICS field devices.
```

Figure 4 - B6 scan result: 10 unit IDs probed, scan complete summary.

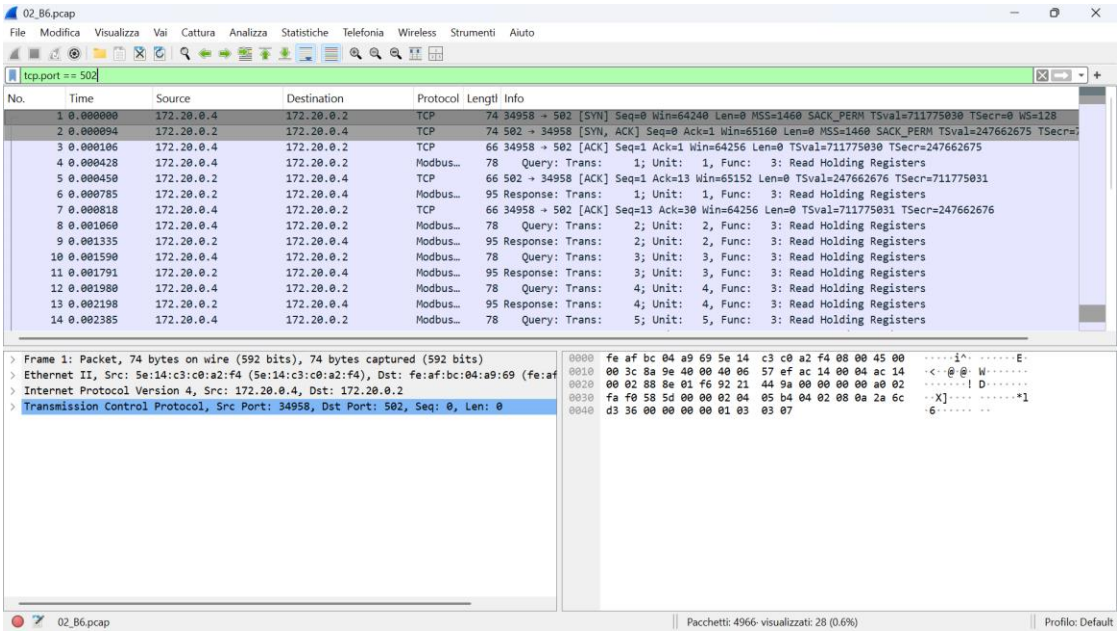

Figure 5 - Wireshark capture of B6: sequential FC03 queries to multiple unit IDs.

Analysis. B6 demonstrates that Modbus has no access control at the unit-ID level. A real-world attacker could scan the entire unit-ID space (247 IDs) in under two minutes and produce a complete device inventory. Note: pyModbus's default server responds to any unit ID; in a real deployment only unit 1 would reply, yielding a realistic discovery of one device.

5. Interruption Attacks

Interruption attacks violate availability. The attacker's goal is to prevent the master from receiving valid, timely responses from the slave.

5.1 T10 - TCP Connection Pool Exhaustion

Description. T10 exploits the finite size of the slave's TCP connection table. The attacker opens many TCP connections to port 502 and holds them open without sending any Modbus data, saturating the pool so the legitimate master cannot connect.

Implementation. The script uses Python's threading module to open 120 TCP connections to 172.20.0.2:502 in parallel (one thread per connection, 30 ms stagger). It performs a connectivity check before, during, and after the attack, then holds the connections for 20 seconds before releasing them.

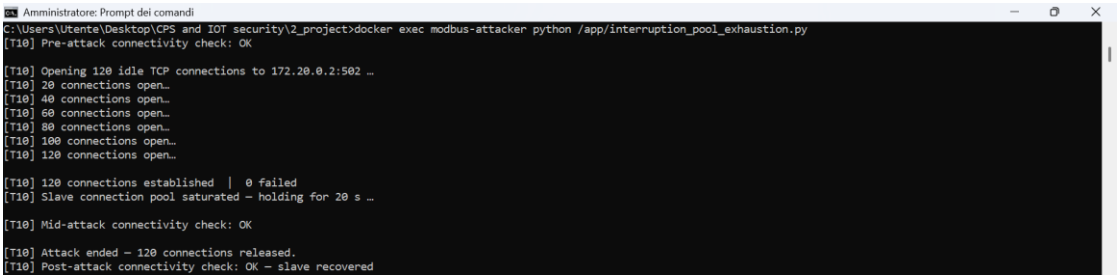

Figure 6 - T10 output: 120 connections established, slave recovered after release.

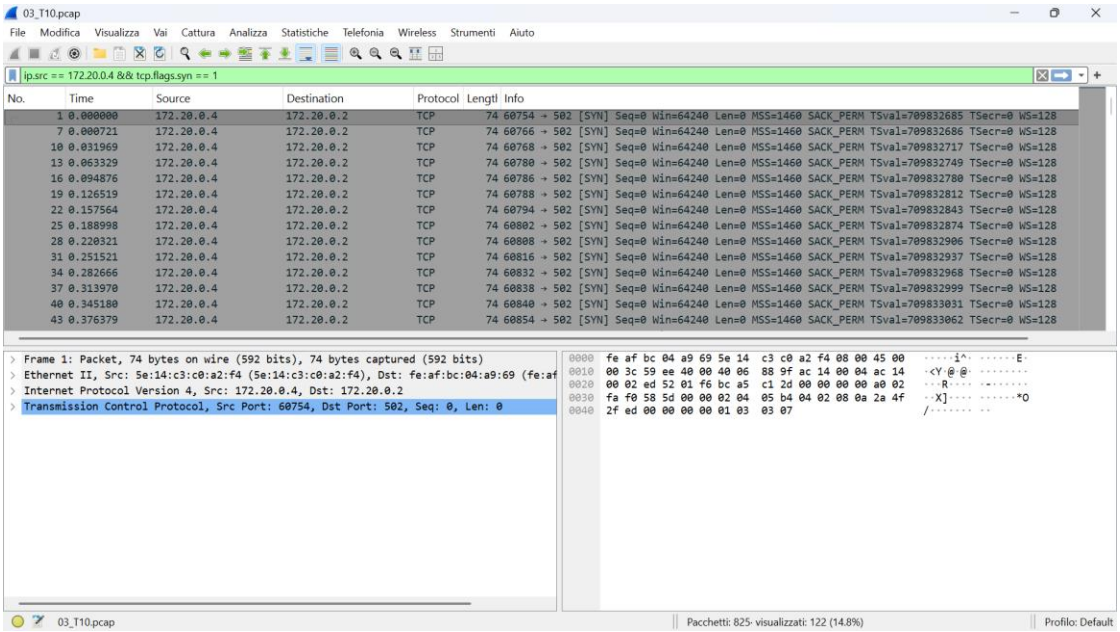

Figure 7 - Wireshark capture of T10: burst of SYN packets from the attacker to port 502.

Analysis. T10 does not require any Modbus knowledge - it is a pure TCP-layer exhaustion attack. The pyModbus asyncio server accepted all 120 connections; production PLCs typically have a limit of 4-32, making this attack far more effective against real hardware. A 20-second interruption in a power grid substation could cause protection relays to trip on stale data.

5.2 T11 - TCP RST Injection

Description. T11 exploits the absence of TCP sequence number authentication. The attacker injects a spoofed TCP RST packet with a valid sequence number into an established Modbus session, causing the slave to immediately close the connection.

Implementation. Scapy is used to perform complete TCP 3-way handshakes directly with the slave, bypassing the OS TCP stack and giving exact control over sequence numbers. For each of 8 cycles: (1) send SYN with random ISN; (2) receive SYN-ACK; (3) complete handshake with ACK; (4) send FC03 request; (5) inject RST at the precise next sequence number, immediately terminating the connection.

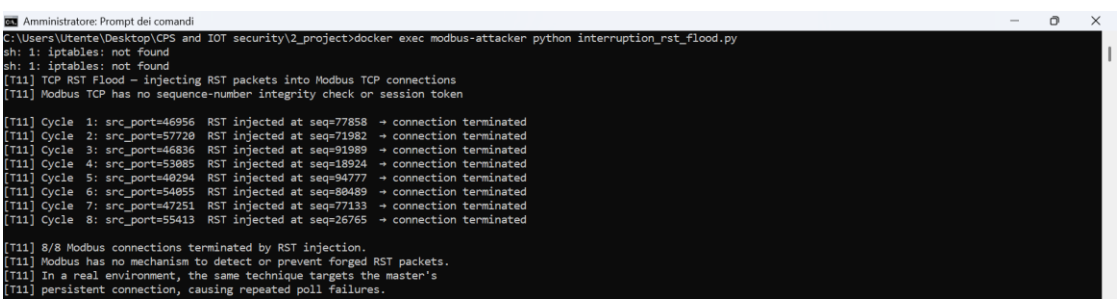

Figure 8 - T11 output: 8/8 RST injections successful, all connections terminated.

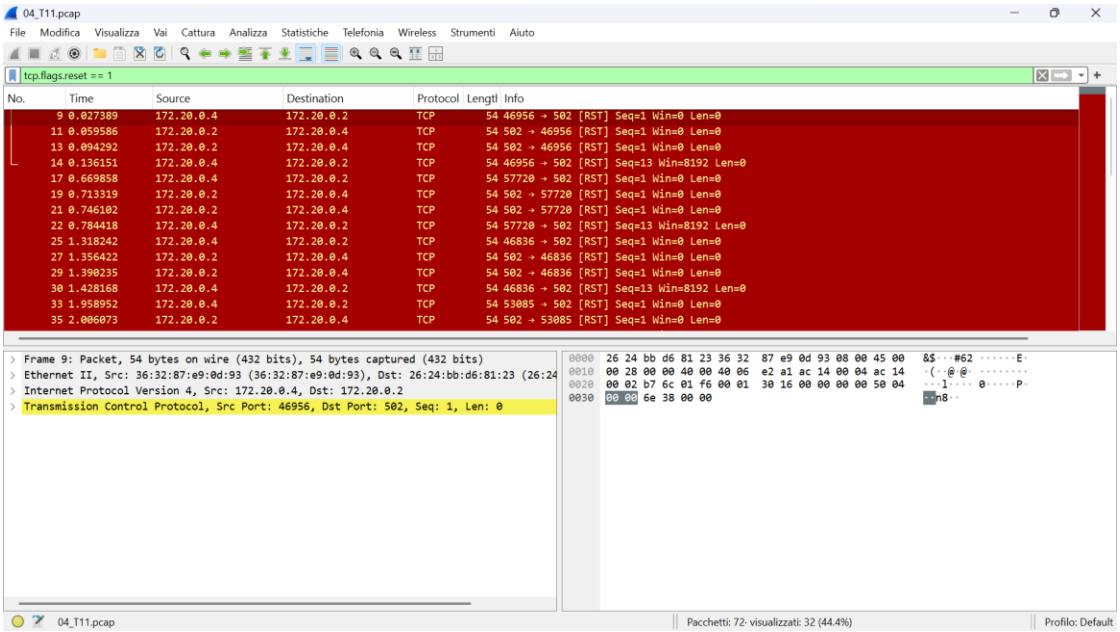

Figure 9 - Wireshark capture of T11 (filter: tcp.flags.reset==1): RST packets visible.

Analysis. TCP RST injection requires only the 4-tuple and a sequence number within the receive window. For a long-lived Modbus connection, the sequence number grows linearly and predictably. Without TCP-MD5 signatures or TLS there is no transport-layer defense. In a real environment with L2 access, repeated RST injections create denial-of-service windows of several seconds per burst, sufficient to disrupt time-critical polling.

6. Modification Attacks

Modification attacks violate data integrity. The attacker intercepts traffic and alters its content before forwarding it, so neither the master nor the slave detects the tampering.

6.1 B14 - Rogue Interloper (Man-in-the-Middle Proxy)

Description. B14 is the most impactful single attack in Table 4, with 15 instances across all four asset classes. The attacker positions itself between master and slave - physically or via ARP cache poisoning - and acts as a transparent TCP proxy. Every Modbus response from the slave is modified before being forwarded to the master, which continues to operate normally, unaware that the sensor readings it receives are falsified.

Implementation. An asyncio transparent proxy listens on port 502 of the attacker container (172.20.0.4). A test master is redirected to connect to 172.20.0.4 instead of 172.20.0.2. The proxy applies tamper_fc03_response() to every FC03 response, doubling all register values before forwarding to the master. This simulates the data falsification an attacker with ARP-poisoned L2 access would perform.

```
Amministratore: Prompt dei comandi
C:\Users\Utente\Desktop\CPS and IOT security\2_project>docker exec modbus-attacker python modification_mitm.py
[B14] Rogue Interloper proxy listening on [('0.0.0.0', 502)]
[B14] Forwarding to real slave at 172.20.0.2:502
[B14] FC03 responses will have ALL register values DOUBLED

[B14] Master connected from ('172.20.0.3', 58044)
[B14] Proxy connected to real slave 172.20.0.2:502
[B14] FC03 response intercepted — original=[8750, 0, 0, 1, 0]  →  sent_to_master=[17500, 0, 0, 2, 0]
```

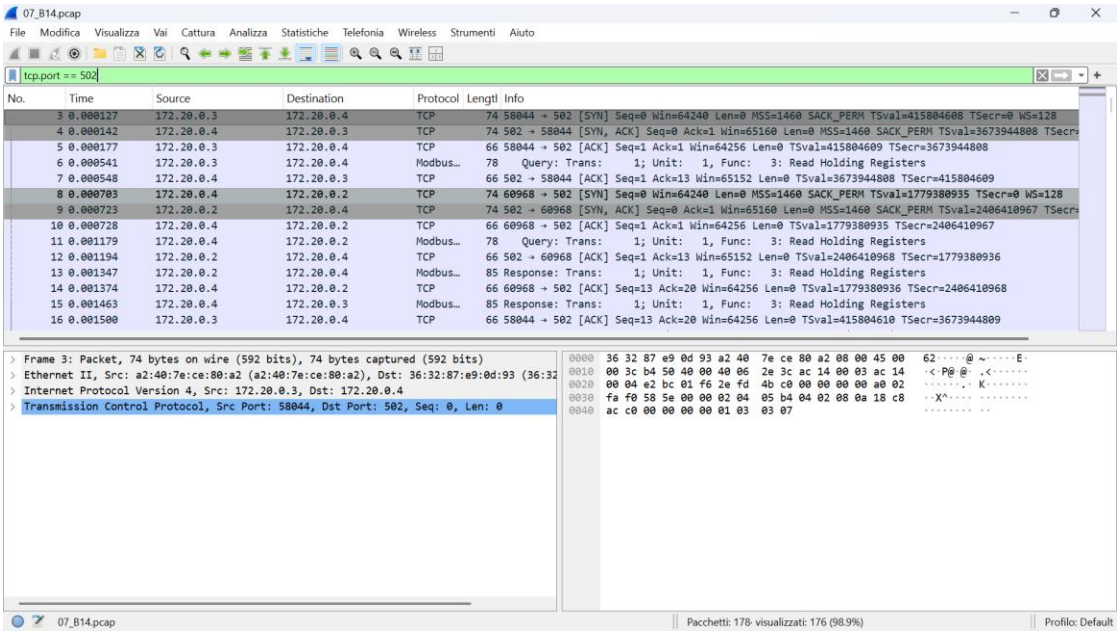

Figure 12 - Wireshark capture of B14: two TCP sessions (master→proxy and proxy→slave) visible.

Analysis. B14 is uniquely dangerous because it is completely transparent to both endpoints: the master receives syntactically valid FC03 responses, and the slave receives syntactically valid FC03 requests. Neither party has any means to detect tampering without message authentication codes. The attacker can selectively modify any subset of registers, inject arbitrary values, or suppress specific function codes - for instance, blocking all write commands while allowing reads, creating a split-horizon view of the plant state.

7. Fabrication Attacks

Fabrication attacks violate authenticity. The attacker generates or replays messages that appear to originate from a legitimate participant.

7.1 B4 - Direct Slave Control (Rogue Master)

Description. B4 exploits the absence of master authentication. Any host that can reach port 502 of a slave can act as a Modbus master: it can read all registers and write any register using FC06 or FC16, while the legitimate master continues to operate concurrently and unaware.

Implementation. The script connects to 172.20.0.2:502 using pyModbus's AsyncModbusTcpClient, reads the current register state, writes malicious values to all five holding registers via FC06 Write Single Register, then reads back the state to confirm the writes succeeded.

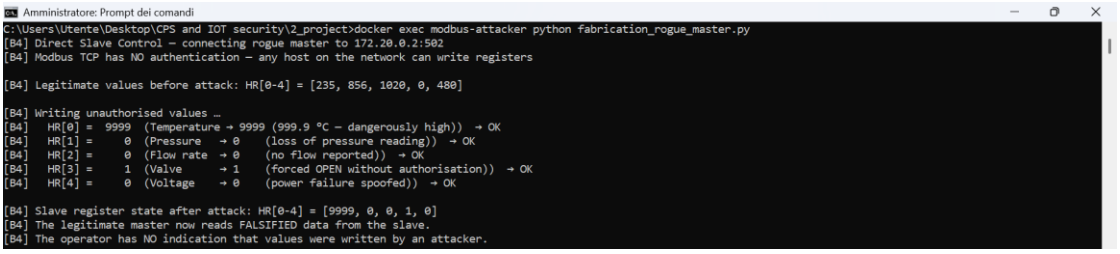

Figure 13 - B4 output: all five registers overwritten; slave confirms falsified state.

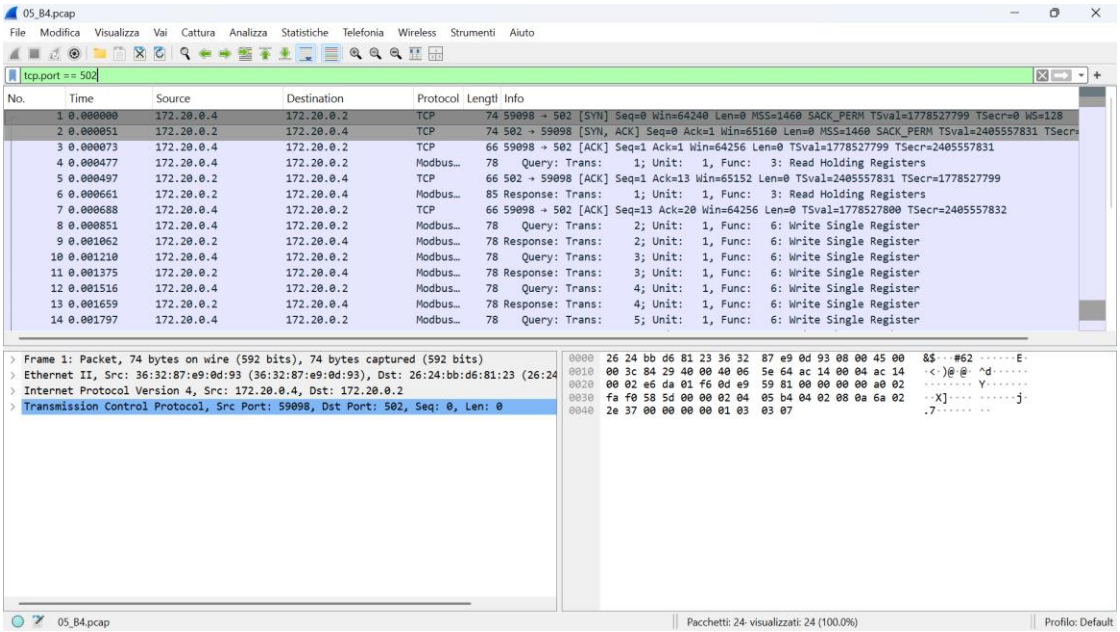

Figure 14 - Wireshark capture of B4: FC03 reads bracketing five FC06 write frames.

Analysis. B4 is the most operationally impactful attack: it directly controls the physical process. Forcing the valve open (HR[3]=1) while reporting 999.9°C (HR[0]=9999) would trigger simultaneous emergency responses - pressure relief, coolant injection, shutdown - while hiding the real sensor readings. The attack requires one TCP connection and executes in under one second. The Wireshark capture shows every FC06 write accepted with a valid response - no exception code, no alert.

7.2 B2 - Response Replay

Description. B2 exploits the absence of message freshness guarantees. The attacker captures a legitimate FC03 response (the 'snapshot') and replays it to the master while the real process state has changed. The master continuously receives stale data and remains blind to the actual conditions.

Implementation. The script performs three phases: (1) reads the current register state as the snapshot; (2) writes new values to simulate a real process change (temperature spike to HR[0]=8750); (3) for 8 cycles separated by 2 seconds, shows what the master would receive (stale snapshot) versus the slave's true current state.

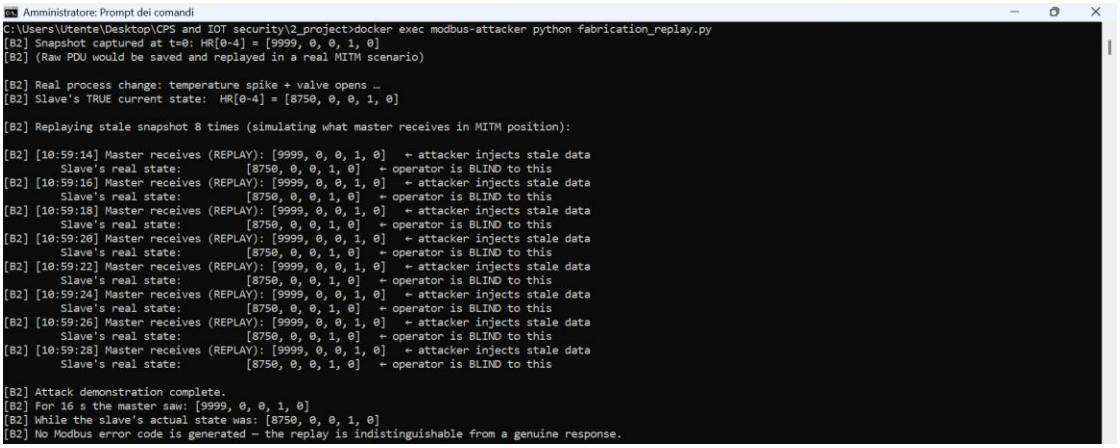

Figure 15 - B2 output: 16 s of stale data replayed; real state diverges silently.

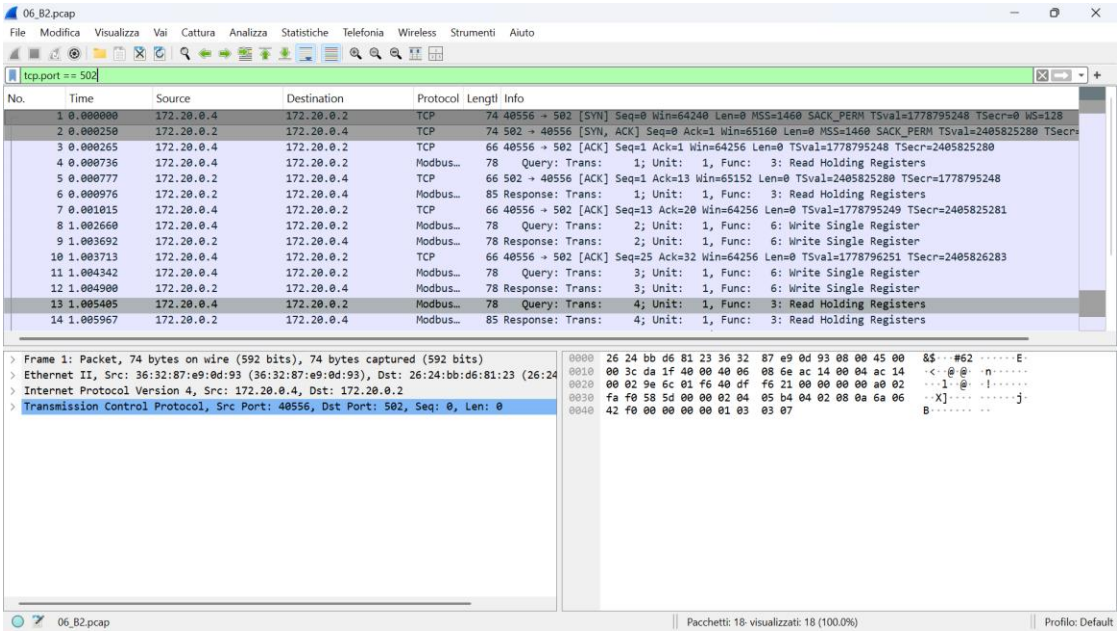

Figure 16 - Wireshark capture of B2: reads and writes confirming the two-phase attack.

Analysis. B2 is particularly insidious because the master's behavior is entirely normal: it receives syntactically valid, correctly structured Modbus responses. The only indication of the attack is the absence of value change over time, which could be attributed to a stable process rather than a replay. In a real MITM position, the attacker would re-inject the captured raw TCP payload at the correct sequence number, making the replay byte-for-byte identical to a genuine response. No Modbus error code is generated.

8. Critical Evaluation

8.1 Severity Comparison

The seven attacks differ substantially in operational impact and in the skill required to execute them. B4 (Rogue Master) and B14 (MITM) are the most dangerous. B4 requires only pyModbus and network reachability and can modify any register in under one second. B14 requires L2 network access (ARP poisoning in a real deployment) but is the only attack that simultaneously affects all four asset classes and remains completely invisible to both endpoints.

T10 and T11 are availability attacks with different persistence characteristics. T10 is detectable by a connection-rate firewall but effective against PLCs with small TCP stacks. T11 creates repeated denial-of-service windows of several seconds per burst; in a real environment where reconnect takes several seconds, this is sufficient to disrupt time-critical polling.

B9 and B6 have the lowest direct impact but the highest strategic value: they provide the reconnaissance foundation for all other attacks. An attacker who has completed B9 and B6 knows the register layout, polling rate, unit ID map, and normal sensor ranges - sufficient to craft a convincing B14 payload or to target the precise register in B4. B2 occupies a unique position: its impact depends entirely on the replayed snapshot; combined with a prior B4 attack, it can lock in arbitrary values indefinitely.

8.2 Testbed Limitations

Three limitations of the Docker-based testbed differ from a real OT environment. First, the bridge driver prevents passive sniffing of master-slave traffic from the attacker container (Section 3.4). B9 and T11 were adapted accordingly. In a real flat L2 OT network with unmanaged switches, this limitation does not apply.

Second, the pyModbus asyncio server accepts an effectively unlimited number of concurrent connections. Real PLCs typically have a limit of 4-32, making T10 far more effective in production. Our testbed shows the mechanism correctly but underestimates the true impact.

Third, register state persists across attack scripts: the B2 snapshot reflects the values left by B4. This models a realistic scenario where a system is attacked sequentially, but the snapshot value [9999, 0, 0, 1, 0] should be read in that context.

8.3 Countermeasures

Network segmentation is the most immediately deployable countermeasure: firewalls that allow only specific master IPs to reach port 502 eliminate B4, T10, T11, and B6 from external attackers.

Modbus/TCP over TLS (RFC 8184 / IEC 62351-3) provides mutual authentication (certificate-based), message integrity (HMAC), and confidentiality (encryption). This eliminates B9, B14, and B2 entirely, and makes T11 infeasible because an RST injection cannot target an encrypted session without breaking the MAC.

Application-layer deep packet inspection - a proxy that enforces an allowlist of (unit ID, function code, register address) tuples - blocks B4 and B6 without requiring protocol changes on legacy devices. Sequence integrity monitoring raises alerts on outlier register values (e.g., temperature > 500 or valve = 2) and would detect the outputs of B4 and B14 without modifying the protocol.

The most practical near-term countermeasure for legacy environments is network segmentation combined with a read-only monitoring proxy. Modbus/TLS adoption is growing in new installations but retrofitting existing plants remains economically challenging.

9. Conclusions

This project implemented a complete Modbus TCP testbed using Docker containers and demonstrated seven attacks spanning all four threat categories of the Huitsing et al. (2008) taxonomy. The experiments confirm the core finding of the paper: Modbus TCP's absence of authentication, encryption, and message integrity makes it fundamentally vulnerable to every class of attack in the taxonomy, using only standard open-source tools and basic network access.

The most significant result is the B14 MITM proxy: the master received register values doubled by the attacker ([17500, 0, 0, 2, 0]) while the slave's actual state was [8750, 0, 0, 1, 0], with no error code, no alert, and no indication of tampering at any protocol layer. Combined with B4, which wrote arbitrary values to all five registers in under one second, these two attacks demonstrate that an adversary with OT network access can both observe and fully control the physical process with commodity tools and minimal expertise.

The path to securing legacy Modbus deployments requires a layered approach: network segmentation as an immediate measure, application-layer proxies as a medium-term solution, and Modbus/TLS as the long-term standard. Without these controls, the protocol's design - openness and simplicity - remains its greatest liability.

References

[1] P. Huitsing, R. Chandia, M. Papa, S. Shenoi. "Attack Taxonomies for the Modbus Protocols." Critical Infrastructure Protection II, IFIP vol. 273, Springer, 2008.

[2] A. Crain et al. pyModbus - A full Modbus protocol implementation in Python. github.com/pymodbus-dev/pymodbus (v3.7.0, 2024).

[3] P. Biondi et al. Scapy - Packet manipulation library. scapy.net (v2.6.1, 2024).

[4] The Modbus Organization. Modbus Messaging on TCP/IP Implementation Guide Rev. 1.0b. modbus.org, 2006.

[5] IEC 62351-3:2014. Power systems management - Data and communications security - Profiles including TCP/IP.